

CI pipeline: what belongs in it, and what does not

A pipeline does not get slow because there are too many tests.
It gets slow because nothing defines what belongs in it.

PRESENTED BY

PRESENTER · TEAM

SCOPE

Every commit to main

What CI answers on every commit

Can main be released with this commit in it?

CI is not a place to run tests. It is that one question, answered on every commit — and the answer is only worth having while a red build means exactly one thing.

Every stage that answers something else makes a red build mean a little less.

THREE WAYS IT GETS DILUTED

Work that answers a different question

Nightly soak runs, full browser matrices

Work nobody reads the answer to

Reports produced, archived, never opened

Work that answers nothing

A suite retried until it happens to pass

The boundary: CI, post-merge, scheduled

IN CI

Failure means: this commit must not merge.

- Build and compile
- Unit and fast integration
- Lint and type checks

AFTER THE MERGE

Only answerable once the merged result exists.

- Deploy to staging
- End-to-end on a real env
- Post-deploy smoke tests

ON A SCHEDULE

The answer does not change commit to commit.

- Full OS and browser matrix
- Licence and CVE audits
- Long soak and fuzz runs

The test: ask what a failure here would mean. If the answer is not "do not merge this commit", it is not a CI stage.

Two policies that keep the signal honest

WHY BOTH EXIST

The gate's only asset is that a red build means something.

Both policies protect that one asset.
Neither of them is really about the test.

01 **A flaky test is a broken test.**

A test that fails without a cause teaches the team to read failures as noise. Quarantine it the day it flakes, then fix it or delete it — a retry is not a fix.

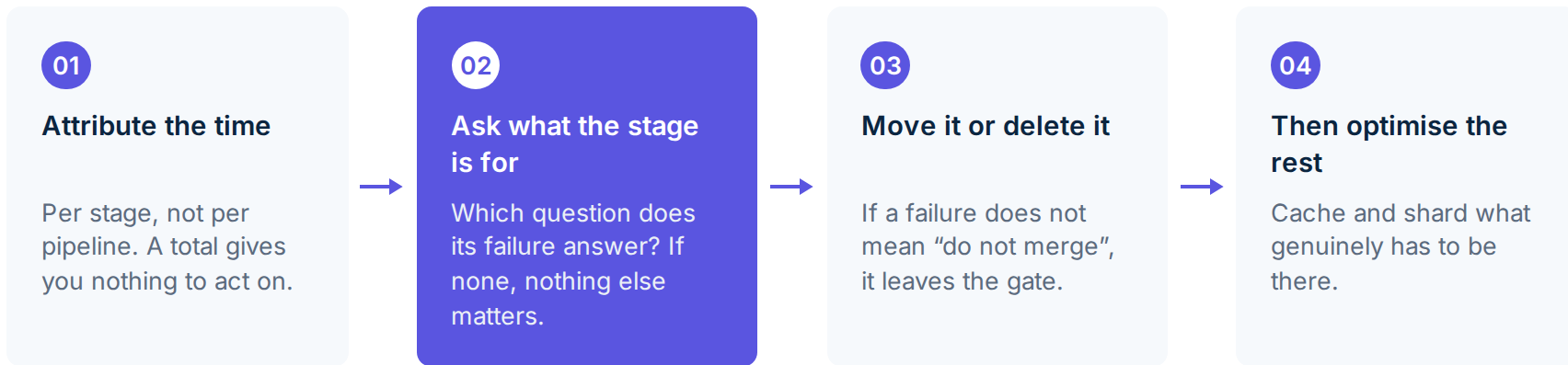
02 **A pipeline nobody trusts gets bypassed.**

Not by rebellion — by batching commits, merging on “probably green”, and handing out overrides. The gate holds only what people will actually wait for.

WHEN IT IS ALREADY SLOW

When the pipeline is already slow

A slow pipeline is a scope problem before it is a speed problem.



Optimising first only makes a stage that should not exist finish sooner.

What we need to decide

Q1

What is our pipeline's one question — and which stages do not answer it?

Q2

Who is allowed to quarantine a flaky test, and how fast?

Q3

What is the longest wait we will defend before a stage leaves the gate?

THE POINT

None of this is a tuning problem until the boundary is written down.

The pipeline is slow because nothing says what belongs in it. Decide that first; the rest really is tuning.

PRESENTED BY

PRESENTER · TEAM